

I. Probleme de tip grila cu un singur raspuns corect

1. Care este o proprietate fundamentală a stivei, indiferent de reprezentarea în memorie?

- A. Accesarea aleatorie a elementelor B. Inserarea și ștergerea se fac la mijlocul structurii
C. Primul intrat este primul ieșit (FIFO) D. Ultimul intrat este primul ieșit (LIFO)

2. Se consideră un arbore binar de căutare în care inserăm, în această ordine, valorile: 50, 30, 70, 20, 40, 60, 80. Ce se va afișa dacă arborele este parcurs în postordine (postorder)?

- A. 20 40 30 60 80 70 50 B. 50 30 20 40 70 60 80
C. 20 30 40 50 60 70 80 D. 80 70 60 50 40 30 20

3. Pentru un arbore binar cu n niveluri, care este numărul maxim posibil de noduri?

- a) n b) $2 \cdot n$
c) $2^n - 1$ d) $1/2$

4. Se consideră algoritmul $\text{cautBin}(\text{st}, \text{dr}, y, x, n)$, unde st și dr sunt numere naturale, x este un vector ordonat crescător cu n elemente numere întregi ($1 \leq \text{st}, \text{dr}, n \leq 10^5$, $x[1], x[2], \dots, x[n]$, $-10^9 \leq x[i] \leq 10^9$ pentru $i = 1, 2, \dots, n$) și y este un număr întreg ($-10^9 \leq y \leq 10^9$, $x[i] \neq y$) care nu face parte din vector. Apelul algoritmului are forma: $\text{cautBin}(1, n, y, x, n)$.

Ce trebuie înlocuit în punctele de suspensie astfel încât algoritmul să returneze poziția celui mai apropiat număr din vector, mai mare decât y ? Dacă un astfel de număr nu există, se returnează -1.

<pre>Algorithm cautBin(st, dr, y, x, n): If st < dr then mij ← (st + dr) DIV 2 If y < x[mij] then Return cautBin(st, mij, y, x, n) Else Return cautBin(mij + 1, dr, y, x, n) EndIf Else EndIf EndAlgorithm</pre>	<pre>A. If y > x[dr] then Return dr + 1 Else Return -1 EndIf C. If y > x[st] then Return st + 1 Else Return -1 EndIf</pre>	<pre>B. If y < x[dr] then Return dr Else Return -1 EndIf D. If y < x[st] then Return st Else Return -1 EndIf</pre>
--	--	--

II. TAD Consideram interfetele pt TAD Stiva si TAD List si o reprezentare pt Stack sub forma de List.

<pre>template <typename T> class Stack { private: ///TODO public: Stack(); ~Stack(); void push(const T& element); void pop(); T& top(); bool isEmpty() const; };</pre>	<pre>template <typename T> class List { private: public: List(); ~List(); void push_back(const T& val); void push_front(const T& val); void pop_back(); void pop_front(); T& front(); T& back(); bool empty() const; int size() const; T& at(int index); void insert(int index, const T& val); void erase(int index); void clear(); };</pre>
---	--

- Implementati metodele clasei Stack
- Simulați comportamentul unei stive de farfurii într-o cantină.

Se consideră o stivă de obiecte de tip `Farfurie`, fiecare având:

- un ID unic (`int id`)
- un nivel de curățenie (`float curatenie`) – valoare între 0 și 1

Cerință:

- Definiți structura `Farfurie`.
- Implementați un program care:
 - adaugă farfurii curate pe stivă folosind metoda `push`
 - extrage farfuriile pe măsură ce sunt luate de clienți (folosind `pop`)

- afișează farfuria de deasupra (t_{op}) doar dacă este suficient de curată (e.g., $curatenie > 0.8$)
- la final, afișează câte farfurii au rămas în stivă.